Nanyang Technological University
College of Computing and Data Science
Team Continuity

Continuity
Release Management Plan

Version: 1.0
Last Modified: October 20, 2026

Document Number: CON-DOC-2026-001
Prepared by: Team Continuity - Jerral Chua Fung Yong, Kataruka Mohika, Jethani Samarth, Sinha Armaan, Jain Sparsh, Faye Chan and Lai Yi
Submitted to: Ajith Santhisenan

APPROVALS

Signature approval of the final document is obtained from Team Continuity's Project Manager and from the Project Sponsor. Until both signatures are recorded, this plan is a draft and does not override the Project Plan.

Submitting Organization's Approving Authority:

Signature	Printed Name	Date	Phone Number
	Kataruka Mohika	21/10/2026	Not Applicable

Continuity Project Manager

Sponsor's Approving Authority:

Signature	Printed Name	Date	Phone Number
	Ajith Santhisenan		Not Applicable

Continuity Project Sponsor, College of Computing and Data Science

REVISION HISTORY

Version	Date	Organization/Point of Contact	Description of Changes
1.0	10/20/26	Jerral Chua Fung Yong	Baseline Version

TABLE OF CONTENTS

1. INTRODUCTION 5
2. REFERENCED DOCUMENTS 7
3. OVERVIEW 8
4. ASSUMPTIONS, CONSTRAINTS, RISKS 10
    4.1. Assumptions 10
    4.2. Constraints 10
    4.3. Risks 11
5. RELEASE APPROACH 13
    5.1. Rationale 13
    5.2. Release Strategy 13
        5.2.1. Release Content 15
        5.2.2. Release Schedule 16
        5.2.3. Release Impacts 17
        5.2.4. Release Notification 18
6. GLOSSARY 19
7. ACRONYMS 21
8. APPENDICES 22
    Appendix A: Build, Package, and Deploy 22
    Appendix B: Rollback Plan 24
    Appendix C: Release Checklist 25

LIST OF FIGURES

Figure 1: Release Schedule 17
Figure 2: Build, Package and Deploy Flow 22

LIST OF TABLES

Table 1: Referenced Documents 7
Table 2: Release Units 9
Table 3: Release-Specific Risks 11
Table 4: Deployment Strategies Considered 13
Table 5: Version History 14
Table 6: Release Content 15
Table 7: Release Schedule 16
Table 8: Release Impacts 17
Table 9: Release Notification 18
Table 10: Acronyms 21

1. INTRODUCTION

Continuity is a web tool that looks at an electronic bill of materials (BOM) as one complete board and flags parts that won't work alongside each other. This plan describes how that tool reaches the people who use it. It applies to every build passed to QA, the lab tutor, the sponsor or end users up to the end of the SC3040 labs. Version v1.3.1 is the final version of this project. Any change after it, whether a fix from feedback or new design work, becomes a new version, numbered as Section 5.2 describes. Version numbers follow the `vMAJOR.MINOR.PATCH` format.

Continuity grows in steps. Each version builds on the last one, adding a few more features each time. One early version is already running at continuity-ui.onrender.com, so people are already using what we ship, and a bad deploy would affect them right away. From 21 October, every update we push also lands inside the two-week DS-12 window, where uptime is being measured.

For every version, the plan answers four questions:

- What does it contain?
- When does it go out?
- How is it built and tested?
- How do we return to the last good version if it goes wrong?

Three documents already cover parts of this ground. The Project Plan sets out the team's roles and schedule, the Quality Plan defines what counts as accepted, and the Change Management Plan controls which changes may go into a version. This plan draws the release-related parts of all three into one place.

Within scope are the constraint engine, the backend (FastAPI and LangGraph), the React frontend, the PostgreSQL database layout, the hosting settings, and the release records kept alongside them. The bench test board and measuring tools are left out, because the team only uses them during testing and keeps them afterwards. Anything after the one-year period during which project evidence is stored is also left out.

The readers we have in mind are:

- the Project Sponsor
- Mohika, who manages the project and its change requests
- Sparsh, who leads development
- Samarth and Faye, who run quality checks
- Armaan, Lai Yi and Jerral, who prepare and repair each version

We revise this plan whenever an approved change request alters a version, and again after feedback from the Lab 5 presentation. Each revision passes through the Change Management Plan and is entered in the Revision History.

On security, the plan points to where passwords and keys are kept, but it never records the values themselves. Those stay in the locked settings on Render and GitLab (DS-13). Results from usability sessions appear only under anonymous participant IDs, in line with the Quality Plan.

2. REFERENCED DOCUMENTS

The Project Plan and the System Requirement Specification (SRS) remain the main sources for requirements. The version numbers come from the team's GitLab commit history. When two documents disagree, we go with the newer one and point out the difference in the section where it comes up.

Step-by-step operating instructions belong in the D10 Deployment Runbook. Appendices A and B give the release-level steps that the runbook expands on. Table 1 lists the documents we used to write this plan.

Table 1: Referenced Documents

Document Name	Document Number	Issuance Date
Continuity Change Management Plan	Version 1.0	October 19, 2026
Design Report on Software Maintainability	Version 1.0	October 19, 2026
Continuity Project Plan	Version 1.1	October 7, 2026
Continuity Software Quality Assurance Plan	Revision 3.0	September 15, 2026
System Requirement Specification (SRS)	Revision 1.0	September 14, 2026
Continuity GitLab commit history (sc3040/TEL1/b)	Exported log, latest commit `d57412f`	October 7, 2026
Continuity README	Repository copy	Not dated
Risk Management Plan (referenced through the Project Plan only)	Version 1.2	Not supplied
CMS Release Plan Template	Template Version 1.0	June 11, 2008

3. OVERVIEW

Continuity reads a plain-English description of a circuit board, picks real parts that can be ordered, and re-checks every rule against every part each time a part is placed. For each result, the engineer can see which field was checked, its value, and where that value came from. The goal is to catch problems a schematic tool can't spot, such as a regulator that will overheat or a part that is out of stock, before anyone builds the board.

One design choice affects every release. A fixed set of rules in the constraint engine decides what is wrong with a board. The language model (GLM-5.2) may only pick a repair from the moves the engine offers, and the engine then checks the board again. The engine has no access to the network, the model or the system clock. Because of this split, a release can change the model, the prompts or the suppliers and the rules will still give the same results. The same holds the other way round. We use this to keep each release small and easy to test.

Work began on 20 August 2026 with seven team members working part time. The code first entered the repository on 3 September, when Sparsh imported the full Continuity source onto the `continuity-import` branch. That commit is v1.0.0. Four more commits added the unchecked flag on verdicts, the analytics API, the analytics dashboard and their tests. MR !1 merged all of this into `main`, which brought `main` to v1.3.1.

At v1.3.1, the prototype ran from start to finish. It had the ten-rule engine, the planner, JLCPCB and Mouser sourcing, the LangGraph repair loop, FastAPI with the event stream, and accounts and projects. The React client already offered CSV export, standalone BOM checks, datasheet upload, memory, the walkthrough and the UC-16 analytics view. v1.3.1 also holds both components the lab manual asks every project to show: an AI component (the language model that reads the brief and picks repairs) and a data analytics component (the validation analytics added in v1.2.0 and v1.3.0).

At v1.3.1, 757 backend tests passed. Backend coverage stood at 91% and engine coverage at 97%, which meets DS-8, and 17 Playwright browser tests also passed. Four things were still missing: the GitLab CI pipeline that enforces coverage, the usability sessions, the bench measurements and the D6–D8 evidence pack.

A single release tag always covers every unit in Table 2. The frontend and backend always ship with the same version number, because the event contract (D2) links them. We don't release the outside services (the JLCPCB and Mouser catalogues and the Z.ai model endpoint), but each release notes which adapter and prompt versions it was tested with.

Table 2: Release Units

Release Unit	Built From	Runs On
Constraint engine (D4)	`backend/continuity/engine`, pure Python with no third-party runtime dependency	Inside the backend; also installable on its own

Release Unit	Built From	Runs On
Backend API and orchestration	FastAPI, LangGraph, Python 3.11+	Render web service, one SSE worker
Frontend	React 19, TypeScript, Vite, Tailwind CSS	Render static site
Database schema	Idempotent statements run at application startup (SRS §12)	Managed PostgreSQL on Neon
Configuration	Environment variables and secrets	Render and GitLab protected settings
Release documents	Release notes, runbook (D10), user guide (D11)	Campus GitLab and the sponsor-nominated Google Drive

4. ASSUMPTIONS, CONSTRAINTS, RISKS

4.1. Assumptions

Campus GitLab holds the master copy of the source code, schemas, prompts, fixtures and tests. This plan follows the repository arrangement recorded in the Project Plan v1.1, Maintainability Report and Change Management Plan.

We assume v1.3.1 stays the final version of the current work. Any future feature that comes out of sponsor feedback starts as a change request. Once the Change Control Board approves it, the feature is built as a new version under the rules in Section 5.2. A fix or refinement becomes v1.3.2 onward, and a new feature starts at v2.0.0.

We expect the GitLab CI pipeline with coverage gates to be running by 21 October, as planned in Project Plan Section 4.6.3. Until then, each release tag needs a manual `pytest --cov` run, with the output saved in the release record.

We also expect the DS-12 observation window to start by 21 October on the prototype that is already online, which puts its end on 4 November. This follows the Project Plan v1.1 and replaces the older date in the Quality Plan, which said the window would start by 4 November.

We expect the team to stay the same. Jerral (126 hours) leads the releases. Armaan, Lai Yi and Jerral advise the Change Control Board on releases, as named in the Change Management Plan. We also expect the lab dates to hold. Lab 5 begins on Thursday 22 October, and its deliverables are due one week later, on Thursday 29 October.

4.2. Constraints

The course schedule fixes the dates. Lab 5 begins on 22 October with the project presentation, the Lab 5 deliverables are due on 29 October, and peer evaluation closes on 6 November. The Project Plan runs to 25 November, which is after the labs end, so this plan works to the lab dates.

The budget pays for one Render web service, one Render static site and one Neon database. We have no separate staging environment, so each release candidate is tested on a team machine and in CI before it goes onto the live site. The SRS also limits the event stream to one application worker and leaves horizontal scaling out of scope. Any deployment method that runs two copies of the app at once is therefore off the table.

SRS Section 5.6 says the hosting plan sleeps when idle, and DS-12 asks for a cold start within 60 seconds. The budget lists Render at SGD 10 a month, and Jerral still needs to confirm which plan we are actually on. DS-12 also caps downtime. At 99% over two weeks, we can afford 336 hours × 1% = 3.36 hours, or about 3 hours 21 minutes, in total. Every restart during a deploy, every failed probe and every missed probe sample uses up part of that.

JLCPCB and Mouser limit how many queries we can send, and the model endpoint must sit in a supported region. For this reason, release checks run on recorded fixtures, with only a short live

smoke test. Campus GitLab can only be reached on the NTU network or through the VPN, so whoever cuts a release needs that access.

4.3. Risks

The risks below are specific to releasing Continuity. The wider project risks (R01 to R09) stay in the Risk Management Plan, which Mohika keeps. Table 3 gives each release risk an owner, a way to reduce it, and what we do if it happens anyway.

Table 3: Release-Specific Risks

ID	Risk	Owner	Mitigation	If It Happens
RL-1	CI coverage gate not running in time, so a release ships without enforced DS-8 checks	Jerral	Pipeline due 21 Oct; from then on, no tag without a passing pipeline	Manual `pytest --cov` run, output attached to the release record, Samarth signs off
RL-2	Deploys use up the 3.36-hour downtime allowance during the DS-12 window	Jerral	v1.3.1 stays frozen during the window; any approved fix gets a health check straight after its deploy	Roll back to the last good tag (Appendix B); log the outage as it happened and keep every sample
RL-3	A schema change can't be undone by rolling back the code	Jerral, Armaan	Only additive, idempotent statements; old code must start against the new schema; reversal tested before any release after v1.3.1	Run the corrective statement from the runbook, then redeploy the previous tag
RL-4	Supplier or model outage during a release stops the live smoke test (links to R06)	Armaan	Release checks run on recorded fixtures first	Ship on fixture evidence; rerun the live smoke test when the source is back; record both
RL-5	A secret is committed or exposed in a release (DS-13)	Jerral	Secret scan in CI; secrets kept only in protected settings	Rotate the credential, remove it from history, retest before the tag is issued again
RL-6	A late change to v1.3.1 before the Lab 5 presentation or submission unsettles the evidence	Mohika	v1.3.1 is frozen; any change needs CCB approval and goes into a new version	Hold the change for v1.3.2 or v2.0.0 and record it as future work
RL-7	Only one person knows the release steps	Jerral	Release notes and Appendix A kept up to date; Armaan shadows the tagging of v1.0.0 to v1.3.1	Armaan runs the release using Appendices A to C
RL-8	A deploy lands during a usability session (DS-15) and changes the build mid-task	Faye, Jerral	Faye shares the session schedule; no deploys during booked slots	Drop that session's result and repeat it on a fixed build

5. RELEASE APPROACH

5.1. Rationale

The lab manual asks for a working prototype at Lab 4 and a final presentation at Lab 5, with all documents checked into the repository. One frozen final version, v1.3.1, gives the tutor, the sponsor and QA a single build to look at, test and grade. The manual's release pattern also suits the time left. Release 1.x covers refinement and bug fixes, and Release 2.0 covers extended features.

The limits in Section 4.2 settle the deployment method. With one hosted environment and one SSE worker, we use a basic deployment, where the new tagged build replaces the old one on Render in a single step. The weak point of this method is that it has no built-in fallback. We cover that with a tested rollback to the previous tag (Appendix B) and with the local-hosting fallback. Table 4 lists the methods from SC3040 Topic 10 that we looked at and why we did or didn't pick each one.

Table 4: Deployment Strategies Considered

Strategy	Adopted	Reason
Basic deployment	Yes	Suits one environment and one worker; simple and cheap. Appendix B covers its main risk.
Multi-service deployment	Not needed	Every unit ships under one tag (Section 3), so no service depends on a different version of another.
Rolling deployment	No	Updates several nodes in batches, and we only run one worker.
Blue-green deployment	No	Needs a second production environment. The budget doesn't include one, and both copies would share one Neon database and schema.
Canary deployment	No	Splits traffic across instances and monitors each one. We have one instance and a handful of users.
A/B testing	No	We aren't running product experiments, and two live versions would muddy the DS-2 determinism evidence.

Two earlier lessons also shaped this approach. The 3 September lab raised the problem of the screen going blank with no message when the API fails, so every release keeps the clear error state, the last sequence number and the retry control (DS-6). The code count came out at about twice the function-point estimate, so we stop adding features at v1.3.1 and keep any later version small.

5.2. Release Strategy

Continuity was built up in stages (a phased function rollout). Every version went to the same small group of users: the team, the sponsor and the usability participants.

Up to v1.3.1, version numbers followed semantic versioning, in the form `vMAJOR.MINOR.PATCH`:

- MAJOR goes up when a change breaks something that already works, such as an event contract change the current frontend can't read.
- MINOR goes up when a merge adds a new feature.
- PATCH goes up for bug fixes, tests and tools that leave the app's behaviour as it was.

The first commit with code, `ef0e632` on 3 September, is v1.0.0. Document uploads, meeting minutes and backlog updates leave the number alone, since they don't touch the code. Table 5 applies these rules to the code commits made so far.

Table 5: Version History

Version	Commit	What Changed	Bump
v1.0.0	`ef0e632`	Full Continuity source imported: backend, frontend, docs and configuration	First release
v1.1.0	`7e67e82`	Verdicts with an unpublished figure now carry an explicit unchecked flag. Older recorded runs still read as plain warnings.	MINOR (new feature, nothing breaks)
v1.2.0	`1ccdfb0`	`GET /analytics` added for DS-16 validation analytics	MINOR (new feature)
v1.3.0	`3ed131e`	Validation analytics dashboard added (UC-16)	MINOR (new feature)
v1.3.1	`1bb235f`	Seeded analytics corpus of 25 runs and the Playwright test suite	PATCH (tests and tools only)

MR !1 (commit `ebc60f9`) brought v1.3.1 onto `main`. The merge added no code of its own, so it keeps the same number. v1.3.1 closes Release 1 and is the final version of this project.

After v1.3.1, we follow the release pattern in the SC3040 lab manual:

- Release 1.x covers refinement and bug fixes. Fixes from feedback become v1.3.2, v1.3.3 and so on.
- Release 2.0 covers extended features. Any new feature or future design work starts at v2.0.0.

None of the five versions in Table 5 has a tag yet. Jerral will push all five tags on 21 October, the day before Lab 5, so the build we present matches a tag. Release candidates for later versions get a suffix such as `v1.3.2-rc.1`. Each build also records its GitLab CI pipeline ID, so any file on the shared drive can be matched to the build that produced it.

We keep the team's trunk-based way of working. Daily work goes into `main` through short-lived feature branches. Every merge request needs approval from someone other than its author, and Sparsh approves anything that touches the engine or the event contract. Up to now, documents

have gone straight onto `main` while the code came in through MR !1. Code keeps arriving through merge requests from here on, so every version maps to a reviewed merge.

For any version after v1.3.1, Jerral cuts a release branch from `main`, for example `release/1.3` for fixes or `release/2.0` for new features. From that point the branch only accepts changes approved for that version, and each one is merged back into `main` as well. CI builds and tests the release branch. Once it passes and the Appendix C checklist is complete, Jerral tags the release candidate. When QA and the CCB accept it, the same commit gets the final version tag and is deployed. The release branch stays open until the next version ships, in case a patch is needed.

A change can enter a version only if it fixes a closed defect or comes from an approved change request. Changes to scope, acceptance criteria or interfaces need the sponsor's written approval through the Change Control Board before they are merged (Change Management Plan, step 5). Everyday defect fixes go through the usual review process.

5.2.1. Release Content

Table 6 sets out the final version and the two kinds of version that can follow it, with what each contains and how it is delivered.

Table 6: Release Content

Version	When	Content	Evidence	Delivered With
v1.3.1 (final release)	Presented at Lab 5 on 22 Oct; submitted with the Lab 5 deliverables on 29 Oct	Ten-rule constraint engine with legal repair set and caps of three repairs per slot and three added slots, planner, JLCPCB and Mouser sourcing, repair loop, event stream, accounts and projects, React client, unchecked flag, analytics API and dashboard, and their tests	DS-1, DS-2, DS-3, DS-4, DS-7, DS-10; DS-8 coverage (91% backend, 97% engine on 7 Oct); DS-16 analytics on the 25-run seeded corpus	Lab 4 documents, including this plan; Lab 5 slides, test plan, test cases and coverage report; all documentation checked into GitLab
v1.3.x (refinement)	After the Lab 5 presentation, as needed	Fixes and refinements raised by the tutor, the sponsor or testing. No new features.	A regression test for each fix	Release notes on GitLab
v2.0.0 (extended features)	Not scheduled	Future design work, such as features outside the current scope or answers to the open issues below	The full Appendix C checklist	A revision of this plan

Some features sit outside v1.3.1: schematic generation, PCB layout and routing, pin assignment, netlists, circuit simulation, manufacturing files and horizontal scaling. Any of these could only arrive in Release 2.0 or later, after an approved change request. Adding a third distributor would also count as a scope change under the Quality Plan.

Four open issues remain in v1.3.1. The first is the export state and CSV columns (DS-11, OI-7), owned by Lai Yi and Armaan. The second is when `relax_requirement` may be used and

whether the engineer is told (OI-21), owned by Sparsh. The third is the 20% threshold for holding back analytics rates, which the code already applies since v1.2.0 (OI-22), also owned by Sparsh. The fourth is the sponsor's confirmation that memory, the walkthrough and standalone BOM checks count as delivered features (OI-19, OI-20), owned by Mohika. v1.3.1 ships with the current behaviour for all four. Mohika will ask for a decision on each after the Lab 5 presentation, and each decision lands in v1.3.x or v2.0.0 depending on whether it adds a feature.

5.2.2. Release Schedule

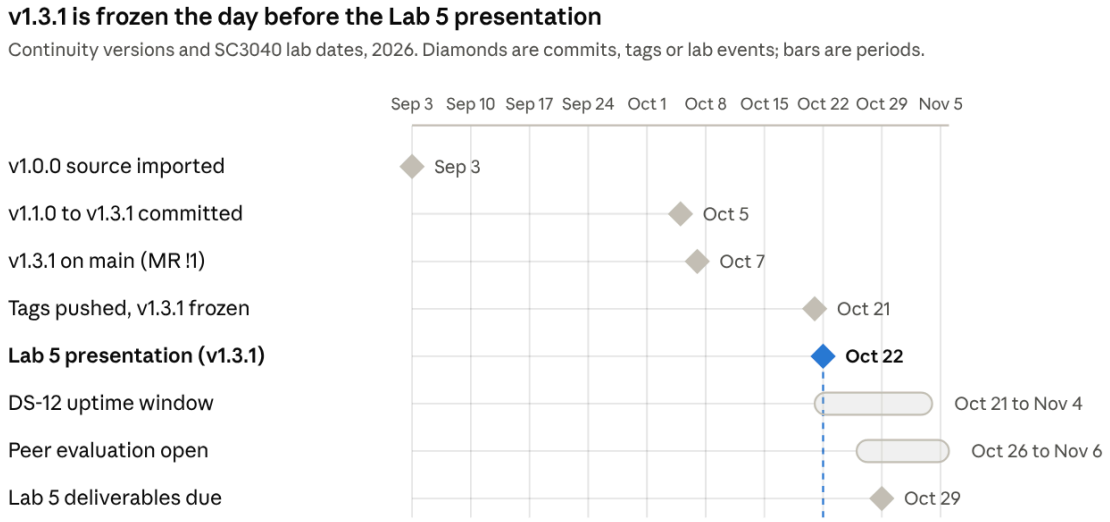
Release dates follow the SC3040 lab schedule. Table 7 lists the dates that matter for releases, and Figure 1 shows them next to the DS-12 window.

Table 7: Release Schedule

Date	Event	Release Action
3 Sep	Continuity source imported (`ef0e632`)	v1.0.0
5 Oct	Unchecked flag, analytics API, dashboard and tests committed	v1.1.0 to v1.3.1
7 Oct	MR !1 merged into `main`	v1.3.1 reaches `main`
21 Oct	Lab 4 deliverables due; scheduled start of the DS-12 observation window	Submit the Lab 4 deliverables. Push tags v1.0.0 to v1.3.1 and freeze v1.3.1 as the final version.
22 Oct (Thu)	Lab 5 begins with the project presentation; this plan is due as a Lab 4 deliverable	v1.3.1 presented
26 Oct to 6 Nov	Peer evaluation open	None
29 Oct (Thu)	Lab 5 deliverables due	All documentation checked into GitLab against v1.3.1
4 Nov	DS-12 window ends	Availability log closed
After Lab 5	Feedback received	v1.3.2 onward for fixes; v2.0.0 for new features

The commit log shows no tags so far. If any code reached `main` after 7 October, it takes the next number under Section 5.2, and that version becomes the one we present and submit.

Figure 1: Release Schedule



The DS-12 window covers both the presentation and the submission date, so any deploy in that period counts against uptime. This is one more reason v1.3.1 stays frozen until the Lab 5 deliverables are in.

5.2.3. Release Impacts

Every release changes the one live site shared by the sponsor, the team and the usability participants. Table 8 shows how each kind of version affects users and systems, and what it gives the project.

Table 8: Release Impacts

Version	Impact on Users	Impact on Systems	Benefit
v1.3.1 (final)	Users keep the live prototype they already use; nothing changes on the day of the presentation	Tags added to existing commits; no redeploy or schema change	One tagged build for the presentation, testing and grading
v1.3.x (refinement)	Issues raised in feedback are fixed; existing accounts and projects carry over	Render redeploy (counts against DS-12 until 4 Nov); additive schema statements only	Feedback closed out without changing features
v2.0.0 (extended features)	New features, once designed and approved	Full Appendix A path; may include schema or event contract changes	Room for future design without touching the graded build

For 24 hours after each production deploy, Jerral watches the three signals named in the SRS: availability (health probes), error rate and run completion. Faye then runs a quick smoke test, taking one brief all the way to CSV export. Any problem is logged in GitLab Issues under the

release tag and discussed at the next weekly QA and defect review. If a failed check puts the Lab 5 presentation or submission at risk, Mohika hears about it straight away.

Probe results are kept as raw logs for the full DS-12 window. A missing sample counts as a failure, and outages caused by our own deploys are logged the same way as any other.

5.2.4. Release Notification

Release news travels through the channels already listed in the Project Plan, and we have added no new ones. Table 9 shows who hears about a release, what they receive, when, and how.

Table 9: Release Notification

Stakeholder	Information Required	Timeframe	Method
Project Sponsor (Ajith)	Release notes, version number, known issues, any DS item still pending	Before the Lab 5 presentation on 22 Oct and with each later version; summary in the fortnightly Thursday status report	From Mohika by email, with files on the sponsor-nominated Google Drive
Lab tutor	Final version number, its tag and the release notes	By the Lab 5 submission on 29 Oct	Campus GitLab, with all documentation checked in
Project Manager and CCB (Mohika)	Proposed scope (closed defects and approved CRs), proposed version number, go/no-go result	Before any tag is pushed	GitLab merge request and Trello card
QA (Samarth, Faye)	Tag, CI pipeline ID, change list, what to test	When a release candidate or final tag is pushed	GitLab release page and Trello
Developers	Freeze notice for v1.3.1, and which fixes are allowed in later versions	21 Oct, and at each branch cut	Trello and the release branch's protected rules
Usability participants	Which build they will use and when	Before each booked session	Faye, using the anonymous-ID process
All of the above, if a release is rolled back	What failed, which version is live now, next step	Within 24 hours, following the escalation route	Mohika phones the sponsor, then sends a written summary

Release notes go on the GitLab release page and are copied to Google Drive. Each set lists the version and the reason for its bump, the CI pipeline ID, the commit range, the closed issues and change requests, any new schema statements, the dependency and prompt versions, the DS evidence produced, known issues, and the steps to roll back.

6. GLOSSARY

Baseline

A version of a requirement, design, plan, build or acceptance target that the team has agreed on and placed under control.

Basic deployment

Swapping the running build for the new one in one step across the whole environment.

Build

A single CI pipeline run that compiles, tests and packages a set of source revisions. Its pipeline ID identifies it.

Code freeze

The moment a release branch is cut. From then on, that release only takes approved fixes.

Cold start

How long the hosted service takes to answer after it wakes from sleep. DS-12 allows up to 60 seconds.

Fixture

A saved supplier response kept in the repository, so tests can replay it without calling the live service.

Health check

A probe that asks the running service whether it is up. A warm service should answer within 500 ms.

Idempotent schema statement

A database change written so that running it a second time on a schema that already has it changes nothing.

Local-hosting fallback

Running Continuity on a team member's machine so a demo can still go ahead when the hosted site is down.

Patch

A release that only fixes defects or adds tests and tools. It raises the PATCH number.

Release

A tested and approved build with its own version tag, handed to people other than the developer who built it.

Release branch

A branch cut from `main` at code freeze. It holds one release and its fixes.

Release candidate (RC)

A tagged build put forward for the go/no-go decision. If it is accepted as it stands, it becomes the release.

Rollback

Going back to the previous working release, and to its schema state where needed, after a deploy fails.

Semantic versioning

Numbering versions as MAJOR.MINOR.PATCH. MAJOR goes up for a breaking change, MINOR for a new feature and PATCH for fixes, tests and tools.

Tag

A fixed name in Git that points to the exact commit that was released.

Unchecked

A rule result meaning the rule could not run because a value it needs was missing. It never counts as a pass.

Vertical slice

One small board taken from start to finish through the real engine, backend, frontend and live data.

7. ACRONYMS

Table 10 lists the acronyms in this plan and what they stand for, in alphabetical order.

Table 10: Acronyms

Acronym	Literal Translation
BOM	Bill of Materials
CCB	Change Control Board
CI	Continuous Integration
CMS	Centers for Medicare & Medicaid Services (template source)
CR	Change Request
CSV	Comma-Separated Values
D1–D12	Project deliverables 1 to 12
DS-1–DS-16	Design Specifications (acceptance criteria) 1 to 16
HTTPS	Hypertext Transfer Protocol Secure
MCP	Model Context Protocol
MPN	Manufacturer Part Number
MR	Merge Request
NTU	Nanyang Technological University
OI	Open Issue
QA	Quality Assurance
RC	Release Candidate
SRS	System Requirement Specification
SSE	Server-Sent Events
UC	Use Case

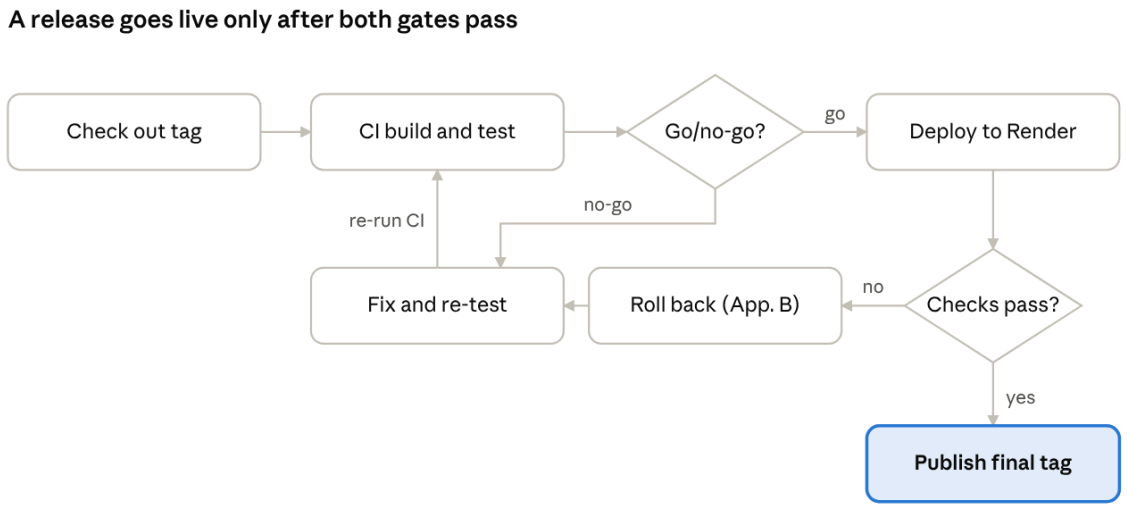
8. APPENDICES

The appendices hold the procedures that Sections 4.3 and 5 refer to. Section 5.2 points to Appendix A for the release steps, Sections 4.3 and 5.1 point to Appendix B for rollback, and Sections 5.2 and 5.2.2 point to Appendix C for the go/no-go check.

Appendix A: Build, Package, and Deploy

Every release takes the same path, whether it is a v1.3.x fix or a new Release 2.0. The build has to be complete and repeatable, and any team member must be able to run it. A build that can't be recreated from its tag doesn't count as a release. Figure 2 shows the path and its two checkpoints, and the steps below explain each stage.

Figure 2: Build, Package, and Deploy Flow



The go/no-go checkpoint is the Appendix C checklist. "Checks pass?" refers to the health check, cold start and smoke run in step 9. Both failure paths end with a fix that goes back through CI. Nobody patches the live site by hand.

1. **Check out.** Make a fresh clone of the release branch at the candidate commit, and install only pinned dependencies.
2. **Build and test in GitLab CI.**
    - Backend tests with `pytest-cov`. The build fails below 80% backend or 95% engine coverage.
    - The JSON Schema check on the event contract.
    - Recorded-fixture regression, plus the 40-board and 25-injection-brief corpora once they exist.
    - Playwright browser tests.

    - The secret scan and the dependency check.
3. **Build the frontend.** Run the Vite production build of the React client from that same commit.
4. **Set the version and tag.** Work out the version number from the changes since the last tag, using the rules in Section 5.2. Tag the release candidate (for example `v1.3.2-rc.1`), draft the release notes, and save the coverage report and test results to Google Drive under the pipeline ID.
5. **Check the package.** Make sure the frontend bundle and the backend image both come from the tagged commit, and that no secrets or local files were included.
6. **Hand off to QA.**
    - Samarth checks the rule, coverage and regression results.
    - Faye runs one full end-to-end test on the candidate, using recorded fixtures for timing.
7. **Go/no-go.** Go through Appendix C. If any required item is left unticked, the deploy stops here.
8. **Deploy to Render.** Confirm the environment variables, deploy the backend web service and static site from the tag, and let the app apply the idempotent schema statements when it starts.
9. **Check after deploy.**
    - Warm health check under 500 ms, and cold start under 60 seconds.
    - One live brief run through to CSV export.
    - Probes still logging.
10. **Publish.** Put the final version tag on the same commit, publish the release notes and send the notifications listed in Section 5.2.4.

Appendix B: Rollback Plan

When a release goes wrong, we roll back first and look for the cause afterwards. Every minute a bad build stays on the only live site uses up availability time. A rollback starts when the health check fails after a deploy, when error rate or run completion gets worse than in the previous release, when a false pass or a silent blank screen shows up, when data is lost or damaged, or when a security exposure is found. Jerral makes the call, with Sparsh for engine or contract faults, and tells Mohika at once. The steps are as follows.

1. **Stop deploys and note the time.** Write down when the fault started. The outage goes into the DS-12 log.
2. **Check the schema.** Appendix C already tests that the previous tag starts against the new schema. If it does, go straight to step 4.
3. **Reverse the schema change if needed.** Export the affected tables, run the corrective statement from the runbook (D10), and confirm the schema matches the previous release record.
4. **Redeploy the previous good tag.** Put the backend and static site back together on Render. Never roll back one of them on its own.
5. **Confirm recovery.** Run the health check and one smoke run, and make sure the probes are green.
6. **Use the local fallback if the host is down.** Start Continuity on a team machine so any planned review or demo can go ahead.
7. **Record and fix.** Open a GitLab issue with the version, cause and outage window. Fix it on the release branch and ship it as the next PATCH version (for example v1.3.2) through the full Appendix A path.
8. **Notify.** Follow the rollback row in Section 5.2.4, and send a written summary within 24 hours.

Appendix C: Release Checklist

Jerral keeps one checklist and applies it to every release. Topic 10 describes this as the core of release management: write the checklist, enforce it, and use the same one for everyone. A release goes ahead only when every item is met, or when the CCB has approved a recorded waiver.

Before the release candidate

- ☐ Scope contains only closed defects and approved change requests (Mohika)
- ☐ Every merge has a non-author review; engine and contract changes are approved by Sparsh
- ☐ Version number matches the changes since the last tag, following the rules in Section 5.2
- ☐ GitLab CI passes: tests pass, backend coverage ≥ 80%, engine coverage ≥ 95%
- ☐ Event-contract JSON Schema check passes
- ☐ Secret scan is clean and dependencies are pinned
- ☐ Regression and injection corpora pass, where they exist
- ☐ Playwright browser tests pass
- ☐ No open false-pass, silent-blank-client or critical security defect
- ☐ Schema statements are idempotent, and the previous tag starts against the new schema
- ☐ Release notes drafted

Go/no-go

- ☐ Samarth (verification) and Faye (validation) sign off
- ☐ Release candidate shared with QA and the CCB before the deploy date
- ☐ Deploy date avoids booked usability sessions, the Lab 5 presentation and the Lab 5 submission
- ☐ Rollback steps (Appendix B) checked against this release's schema changes

After deploy

- ☐ Warm health check under 500 ms; cold start under 60 seconds
- ☐ Live smoke run from brief to CSV export passes
- ☐ Availability probes logging
- ☐ Final version tag applied and release notes published on GitLab
- ☐ Evidence saved to Google Drive under the pipeline ID; second copy kept for v1.3.1, the final version
- ☐ Notifications sent as in Section 5.2.4